

Intermediate HuggingFace

Custom training loops, DPO alignment, model quantization, ONNX export,
deployment

01 Custom Training with Trainer API

The Trainer API handles the training loop, evaluation, checkpointing, and logging — you only define the model, dataset, and metrics.

```
# Fine-tune a classifier with Trainer
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)

from datasets import load_dataset
import numpy as np
from evaluate import load as load_metric

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(model_name, num_labels=2)

# Load and tokenize dataset
dataset = load_dataset("imdb")
def tokenize(examples):
    return tokenizer(examples["text"], truncation=True, padding="max_length")
tokenized = dataset.map(tokenize, batched=True)

# Define metrics
accuracy = load_metric("accuracy")
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return accuracy.compute(predictions=preds, references=labels)

# Training config
args = TrainingArguments(
    output_dir="./results",
    num_train_epochs=3,
```

```

        per_device_train_batch_size=16,
        per_device_eval_batch_size=16,
        eval_strategy="epoch",
        save_strategy="epoch",
        load_best_model_at_end=True,
        metric_for_best_model="accuracy",
        logging_dir="./logs",
        report_to="wandb", # track in Weights & Biases
    )

    trainer = Trainer(model=model, args=args,
                      train_dataset=tokenized["train"],
                      eval_dataset=tokenized["test"],
                      compute_metrics=compute_metrics)

    trainer.train()

```

02 DPO — Direct Preference Optimization (Alignment)

DPO is a simpler alternative to RLHF for aligning LLMs to human preferences. Instead of training a reward model, DPO directly trains on pairs of (chosen, rejected) responses. The TRL library makes this easy.

```

# DPO fine-tuning with TRL
from trl import DPOTrainer, DPOConfig
from transformers import AutoModelForCausalLM, AutoTokenizer
from datasets import Dataset

# DPO dataset format: prompt + chosen + rejected
dpo_data = [
    {"prompt": "Explain neural networks simply:",
     "chosen": "Neural networks are layers of math...", # preferred
     "rejected": "I cannot explain that."}, # not preferred
    # ... more examples
]

model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B")
ref_model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B")

training_args = DPOConfig(
    output_dir="./dpo_output",
    num_train_epochs=1,
    beta=0.1, # KL divergence penalty – higher = stay closer to ref model
)

trainer = DPOTrainer(
    model=model,
    ref_model=ref_model,
    args=training_args,

```

```
train_dataset=Dataset.from_list(dpo_data),
)
trainer.train()
```

03

Model Quantization — Run Large Models on Small Hardware

Quantization reduces model precision (float32 → float16 → int8 → int4), dramatically reducing memory and increasing inference speed with minimal quality loss.

Format	Memory Reduction	Quality Loss	Library	Use For
float32 (full)	Baseline	None	Default	Training
float16 / bfloat16	50%	Negligible	torch.float16	Inference on GPU
int8 (W8A8)	75%	Tiny	bitsandbytes	CPU/GPU inference
int4 (QLoRA)	87.5%	Small	bitsandbytes	Fine-tuning + inference on consumer GPU
GGUF (CPU)	Variable	Small	llama.cpp	Local CPU inference (MacBook, laptop)
ONNX	Variable	None	optimum	Cross-platform, mobile, edge deployment

```
# Load in 4-bit for inference
from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig
import torch

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16,
    bnb_4bit_use_double_quant=True, # nested quantization
    bnb_4bit_quant_type="nf4" # NF4 is better for LLMs
)

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-3.2-8B-Instruct",
    quantization_config=quantization_config,
    device_map="auto")

# Llama 8B normally requires ~16GB VRAM
# With 4-bit: only ~5GB VRAM — runs on RTX 3080/M2 Mac
```

04

Publishing Models to HuggingFace Hub

```
# Push fine-tuned model to Hub
from huggingface_hub import HfApi
```

```

# Login (once)
# huggingface-cli login # or:
from huggingface_hub import login
login(token="hf_YOUR_TOKEN")

# Save and push model
model.push_to_hub("your-username/my-kyc-extractor")
tokenizer.push_to_hub("your-username/my-kyc-extractor")

# Push with model card (README)
api = HfApi()
api.upload_file(
    path_or_fileobj=b"# My KYC Model\nFine-tuned for KYC extraction...",
    path_in_repo="README.md",
    repo_id="your-username/my-kyc-extractor")

# Load from Hub anywhere
model = AutoModelForCausalLM.from_pretrained("your-username/my-kyc-extractor")

# PEFT / LoRA adapters only (much smaller files)
peft_model.push_to_hub("your-username/my-lora-adapter")

# Then load: PeftModel.from_pretrained(base_model, "your-username/my-lora-adapter")

```

05 Inference Optimization for Production

```

# vLLM – high throughput serving
# Install: pip install vllm

# Serve any HuggingFace model with OpenAI-compatible API
# python -m vllm.entrypoints.openai.api_server \
#     --model Qwen/Qwen2.5-Coder-7B-Instruct \
#     --port 8000

# Then call it exactly like OpenAI:
from openai import OpenAI
client = OpenAI(base_url="http://localhost:8000/v1", api_key="dummy")

response = client.chat.completions.create(
    model="Qwen/Qwen2.5-Coder-7B-Instruct",
    messages=[{"role": "user", "content": "Write a Python sort function"}]
)

# vLLM features: PagedAttention (2x throughput), continuous batching,
# tensor parallelism (multi-GPU), streaming, OpenAI-compatible API

```